Documentación Técnica IOT

PEDRO GALERA FERNANDEZ

Resumen

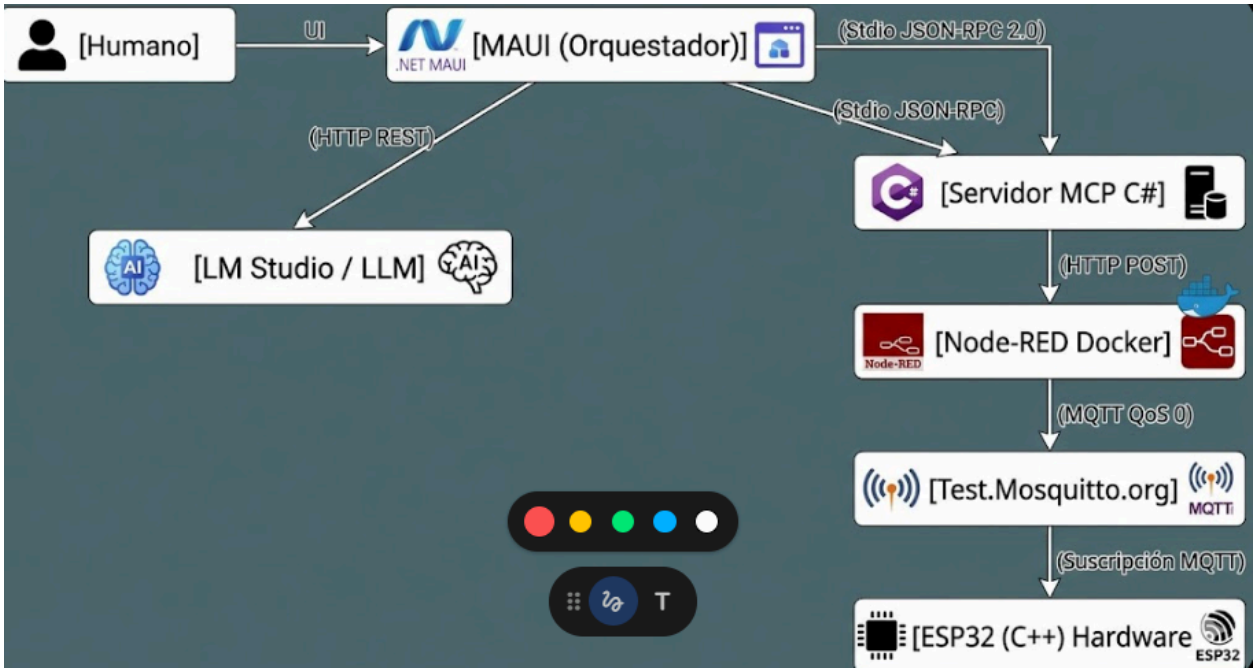
Nuestro proyecto consta de 2 proyectos de C#, un proyecto MAUI para cliente con el que el usuario interactúa con el modelo y el mcp, un servidor MCP para interactuar con Node-RED.

Nuestro propósito a través de un modelo , con una petición http ser capaces de interactuar con un dispositivo IoT. Logramos esto utilizando Node-RED, junto a un broker público de protocolo MQTT.

Arquitectura general

En nuestra arquitectura, nuestra aplicación MAUI actúa como Orquestador, interactúa con el servidor MCP y LM Studio, manteniendo el modelo en un sitio y nuestro servidor mcp en otro.

El servidor MCP es el que interactúa con Node-RED, el encargado de publicar el mensaje que consumirá nuestro ESP32 de un broker público



MAUI

En MAUI hemos respetado el patrón MVVM, utilizando MainViewModel.

Lo más importante de este proyecto es , como se ha mencionado anteriormente, que mantenemos dos servicios para interactuar con el modelo y el servidor MCP:

- LlmService.cs: Se conecta al modelo LLM.
- McpClienteService.cs: A través de ProcessStartInfo, inicia el servidor MCP como proceso secundario. Luego, MAUI se apropia de los flujos estándares para comunicarse localmente con él (StandardInput/Output).

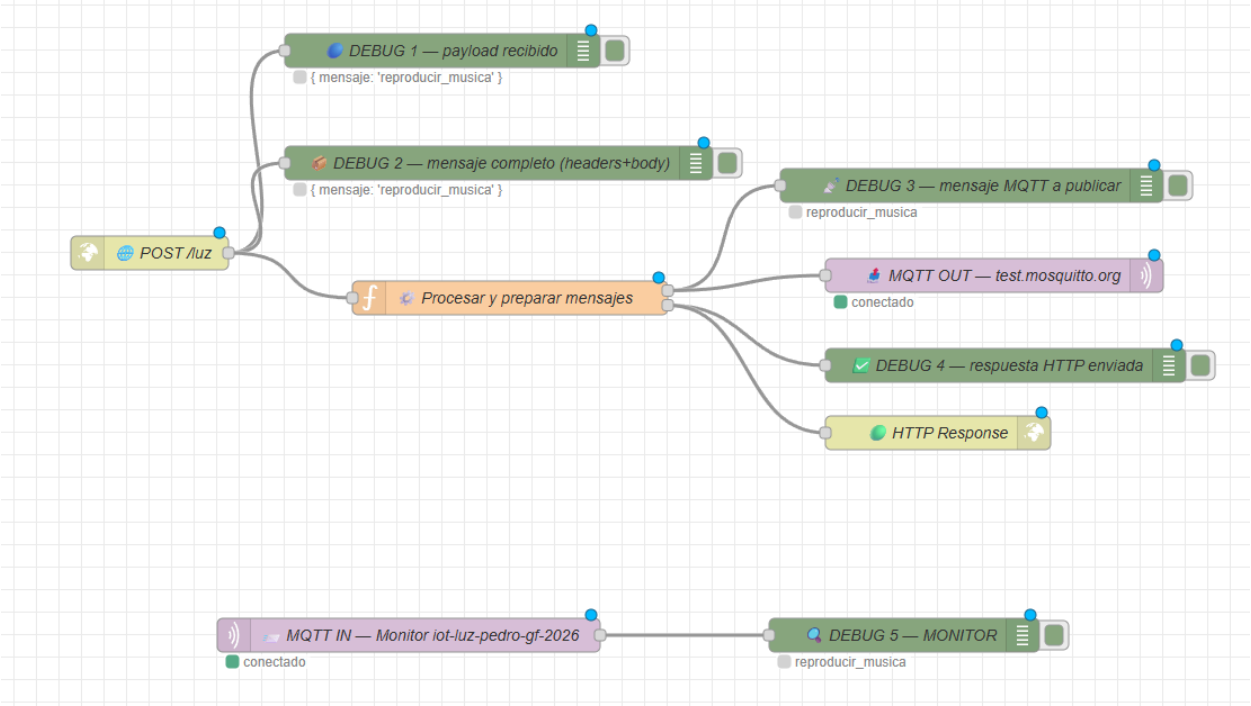
Model Context Protocol

Una aplicación de consola con ciclo de vida permanente basada en eventos.

1. Transporte: tenemos un flujo de lectura asíncrona sobre Stdio con un StreamReader serializado.
2. Especificamos las normas
3. Registro de eventos (tool/list): Declaramos un esquema de funciones (encender_roja,reproducir_musica)
4. Capa de abstracción REST (tools/call): El servidor no envía los comandos al dispositivo IoT, ni habla MQTT. Utilizamos la interfaz HttpClient para crear un PostAsync para mandar de forma segura en JSON la instrucción a la ruta /luz, en Node-RED

Node-RED

Utilizamos Node-RED como un traductor de HTTP a MQTT

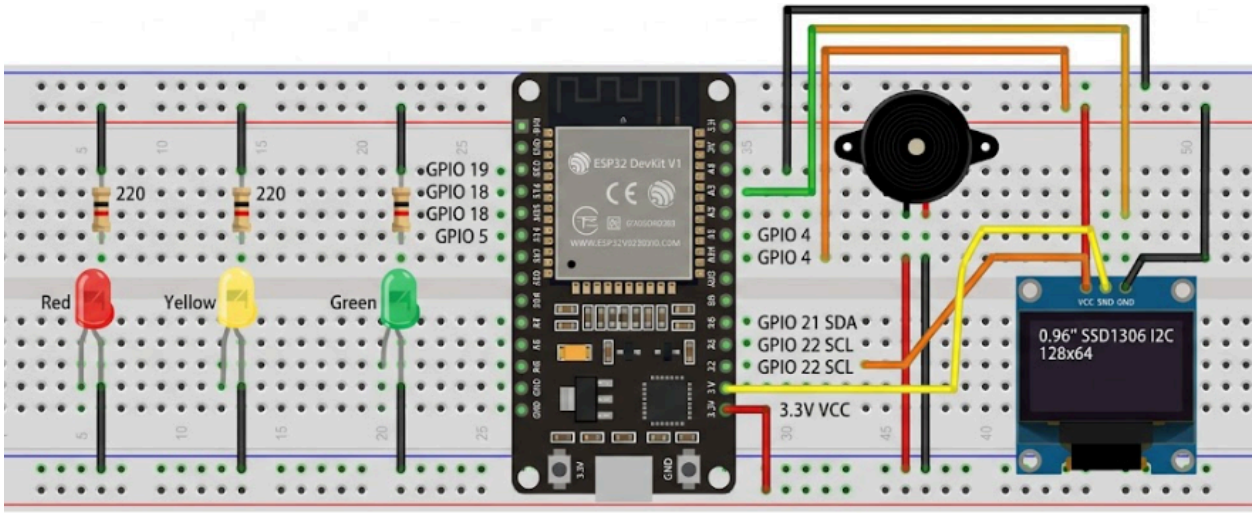


- Comenzamos con un bloque HTTP in que abre el socket y recibe la petición HTTP del servidor MCP
- El bloque de función nos prepara valida el mensaje recibido y prepara dos mensajes para enviar: un mensaje HTTP y otro MQTT
- El mensaje MQTT se publicará en un broker público Mosquitto (test.mosquitto.org) a través del topic iot-luz-pedro-gf-2026.
- El otro mensaje se mandá al cliente como mensaje de confirmación y se cierra la conexión con HTTP Response.
- Con el bloque MQTT IN vemos el mensaje recibido.

En todo momento vemos el mensaje gracias a los bloques de DEBUG.

Hardware IoT : ESP32

En nuestro caso, contamos con un ESP32



En la protoboard tenemos 3 leds conectados a los pines 5,18,19 con sus respectivas resistencias.

También poseemos un buzzer o zumbador para reproducir sonidos.

Por último, tenemos una pantalla que utilizamos como log para mostrar los mensajes recibidos.

Código IoT

El sistema que se utiliza es subscribe-publisher. El dispositivo se suscribe a un topic iot-luz-pedro-gf-2026 , y cuando este recibe un mensaje lo consume el dispositivo.

Esto lo logramos utilizando las bibliotecas WifiClient.h para el stack TCP/IP y PubSubClient.h para la conectividad MQTT.

Una vez dentro, se programan las funciones para encender luces o reproducir las músicas.

Disponemos de un Callback que se bloquea de forma eficiente y solo se dispara cuando recibe un input entrante.

Despliegue

Para ejecutar la aplicación, debemos disponer de Node-RED (en mi caso utilizo Docker), el programa y el dispositivo IoT.

Con solo iniciar la aplicación se lanzará el servidor MCP. Los prompts y rutas están hardcodeadas, por lo que si es necesario modificarlas se debe hacer desde el código (las que se encuentran en el código son las que se crean por defecto).